

Community Events Portal

Lightweight Node.js + Express community events portal for creating and registering volunteers for local events. The project includes server routes, Sequelize models, EJS views and a static frontend in **Public**.

Quick facts

- Tech: Node.js, Express, Sequelize, EJS
- Entry file: [index.js](#)
- Default port: **7000** (set in [index.js](#))

Prerequisites

- Node.js 14+ (recommended)
- npm (comes with Node.js)

Install project

1. Clone or copy the repo to your machine.
2. From the project root run:

```
npm install
```

Notes: this repository depends on **sqlite3** and ships with **nodemon** in **dependencies**. **npm start** runs **nodemon index.js** by default (see **package.json**).

Running the app

- Development (uses **nodemon**):

```
npm start
```

- Production (direct node):

```
node index.js
```

Open <http://localhost:7000> in your browser after the server starts.

Common mistake: do not run **node start** — use **npm start** instead.

Database configuration

By default the project uses an in-memory SQLite database (see [Utils/SqlDb.js](#)). This is useful for demos but data will be lost when the server restarts.

Change to a file-based SQLite DB (persistent) by editing [Utils/SqlDb.js](#) to:

```
const { Sequelize } = require('sequelize');
const sequelize = new Sequelize({ dialect: 'sqlite', storage:
  './database.sqlite' });
module.exports = sequelize;
```

Or configure MySQL/Postgres via environment variables, for example:

```
const sequelize = new Sequelize(process.env.DATABASE_URL, { dialect: 'postgres'
});
```

After changing DB config restart the server — [sequelize.sync\(\)](#) in [index.js](#) will create the tables if they don't exist.

Seed data

On server start [index.js](#) runs a seeding block that creates several sample events and a demo user (email: [reddygovardhan6826@gmail.com](#)). Remove or guard this block for production use (it's inside the [app.listen](#) callback near the bottom of [index.js](#)).

Email sending

- [Utils/SendMail.js](#) calls a remote Azure Logic App endpoint to actually send mail. [Utils/SendGroupMail.js](#) wraps that and iterates recipients.
- If you want to replace this with SMTP, edit [Utils/SendMail.js](#) to use [nodemailer](#) and set SMTP credentials in environment variables.

Endpoints and usage

Below are the most-used routes and example requests.

- GET / — main static page
- GET /game — serves the game
- GET /location — returns geoip-based location (used by community pages)

User flows (rendered EJS pages):

- GET /UserLogin — login form
- POST /UserLogin — body: { [email](#), [password](#) } — logs in user (server emulates session with [Utils/LocalStorage.js](#))
- GET /UserRegister — registration form
- POST /UserRegister — registers a new user (fields come from the HTML form)

- GET /UserHome — protected user home (relies on `Utils/CheckUserAuth.js`)

Events API (JSON)

- POST /events/register — register a user for an event.
 - Body (guest): `{ eventId, firstName, lastName, inputEmail, phoneNumber, inputAddress, inputCity, inputZip, inputState, isSubscribed }`
 - If logged in, the server links the logged-in user (no need to pass email/password).
- GET /events/:id — returns event JSON. Add `?users=1` to include registered users.
- GET /events?city=Hyderabad&status=open — query events by city and status.
- POST /events — create a new event (admin). Example JSON body:

```
{
  "Title": "Community Cleanup",
  "Description": "Help clean the park",
  "EventDateTime": "2026-04-22",
  "MaxNoOfSeats": 100,
  "Location": "Park Entrance",
  "City": "Hyderabad",
  "State": "Telangana"
}
```

Server will send group email to subscribed users when a new event is created (via `Utils/SendGroupMail.js`).

Troubleshooting

- If server immediately exits with errors, check the terminal output. Common issues:
 - `Error: Cannot find module 'sqlite3'` — run `npm install`.
 - Running `node start` instead of `npm start` — use `npm start` or `node index.js`.
 - If emails fail, verify network access to the Azure Logic App URL or update `Utils/SendMail.js` to a local SMTP.

Helpful edits you might make

- Persist DB between restarts: change `Utils/SqlDb.js` to use file-based SQLite as shown above.
- Use an environment variable for port: replace `const PORT = 7000` in `index.js` with `const PORT = process.env.PORT || 7000`.
- Replace `Utils/LocalStorage.js` session emulation with `express-session` for real sessions.

Development notes

- Views (EJS) live in `views/` and static frontend in `Public/`.
- Data access models are in `DataAccess/`.

Screenshots

Screenshots of the main pages (captured with a headless browser) are in the `/screenshots` folder.

 Home

 Game

 Community Home

 User Login

 User Register

Contributing

- Fixes and documentation improvements welcome. Open issues or PRs.

License

- Add a LICENSE file or choose an open-source license.

If you want, I can: add a `serve` script to `package.json` (node `index.js`), update `index.js` to use `process.env.PORT`, or change `Utils/SqlDb.js` to use file-based SQLite — tell me which and I'll apply the change.